

CHESS TOURNAMENT MANAGER

Product Requirements Document (PRD) — v1.0 MVP

SPECIFICATION

PRODUCT TYPE

Web Application

TARGET STACK

React / Node / Postgres

PRIMARY METRIC

500 Players / 50 Rounds

DOCUMENT DATE

March 2026

1. EXECUTIVE PRODUCT OVERVIEW

Chess Tournament Manager is a clean, lightweight web application designed for tournament organizers conducting school, college, local club, and amateur FIDE-style chess events. The platform removes administrative overhead by automating pairing logic, result tracking, standings calculation, and performance rating estimations without relying on bloated interfaces or excessive visual clutter.

Core Design Goal: Prioritize structural simplicity, strict computational correctness (especially regarding FIDE tiebreaks and Swiss pairing constraints), and high operational reliability over heavy UI framework overhead.

2. TARGET PERSONA & USER REQUIREMENTS

Tournament Organizer

- Needs rapid multi-player onboarding (bulk or single entry).
- Requires strict algorithmic automated pairings (Swiss, Round Robin, Knockout, Arena).
- Demands effortless result entry and instant standing updates.
- Wants zero manual tiebreak or performance calculations.
- Controls round advancement, bye management, and forfeits.

Players & Spectators

- Requires clear round pairing boards (Opponent, Color, Board #).
- Views live, real-time updated leaderboards and results.
- Accesses personal performance metrics and FIDE rating deltas.
- Mobile-responsive access to active tournament statuses.
- Inspects round-by-round tournament match history.

3. END-TO-END OPERATIONAL FLOW

- Tournament Creation:** Define Name, Location, Organizer, Format, Number of Rounds, Rating System, and Time Control.
- Player Roster Management:** Input player details, FIDE IDs, ratings, age, federation, and club/institutional affiliations.
- Engine Initialization:** Lock roster, initialize seeds by rating, and trigger Round 1 pairing algorithm.
- Round Execution & Result Entry:** Record match outcomes (1-0, 0-1, ½-½, Byes, Forfeits) with live validation.
- Standings & Re-Pairing:** System auto-calculates Buchholz/Sonneborn-Berger tiebreaks and generates subsequent round pairings.
- Tournament Conclusion:** Lock final state upon round completion, display podium, and generate final performance rankings.

4. FUNCTIONAL REQUIREMENTS: SETUP & ROSTER MANAGEMENT

4.1 Tournament Configuration Module

The organizer interface provides a streamlined creation form with strict client-side and server-side validation:

FIELD NAME	TYPE / OPTIONS	CONSTRAINT / VALIDATION
Tournament Name	Text Input	Required (e.g., "FRCRCE Chess Championship 2026")
Location / Host	Text Input	Required (e.g., "Mumbai, India")
Organizer Name	Text Input	Required (e.g., "College Chess Committee")
Tournament Type	Dropdown	Swiss System, Round Robin, Arena, Knockout
Number of Rounds	Numeric Input	Integer min 1, max 50 (Organizer controlled)
Time Control	Select / Text	Optional (e.g., "10+0 Rapid", "90+30 Classical", "3+2 Blitz")
Rating System	Dropdown	FIDE System / Custom Rating

4.2 Player Roster Schema & Data Table

Player inputs capture essential identity metrics to allow correct rating calculations, color history tracking, and seeding:

# SEED	PLAYER NAME	FIDE ID	RATING	AGE	FED / CLUB	ACTIONS	
1	Magnus Carlsen	1503014	2837	35	NOR / Offerspill	Edit	Delete
2	Hikaru Nakamura	2016192	2816	38	USA / Chess.com	Edit	Delete
3	Fabiano Caruana	2020000	2779	34	USA / Saint Louis	Edit	Delete
4	Vidit Gujrathi	5029410	2720	31	IND / PSPB	Edit	Delete

5. SUPPORTED TOURNAMENT FORMATS

A. Swiss System

Non-eliminary system pairing players with similar scores. Strict constraints prevent repeat matchups, enforce color balance (maximum 2 consecutive same colors; difference $|W - B| \leq 2$), and assigns half-point/full-point byes for odd roster counts.

B. Round Robin

All players face every other player once. Automatically generates complete N-1 rounds (even) or N rounds (odd) using standard Berger table round-robin rotation. Automatically handles odd-player byes seamlessly.

C. Knockout Bracket

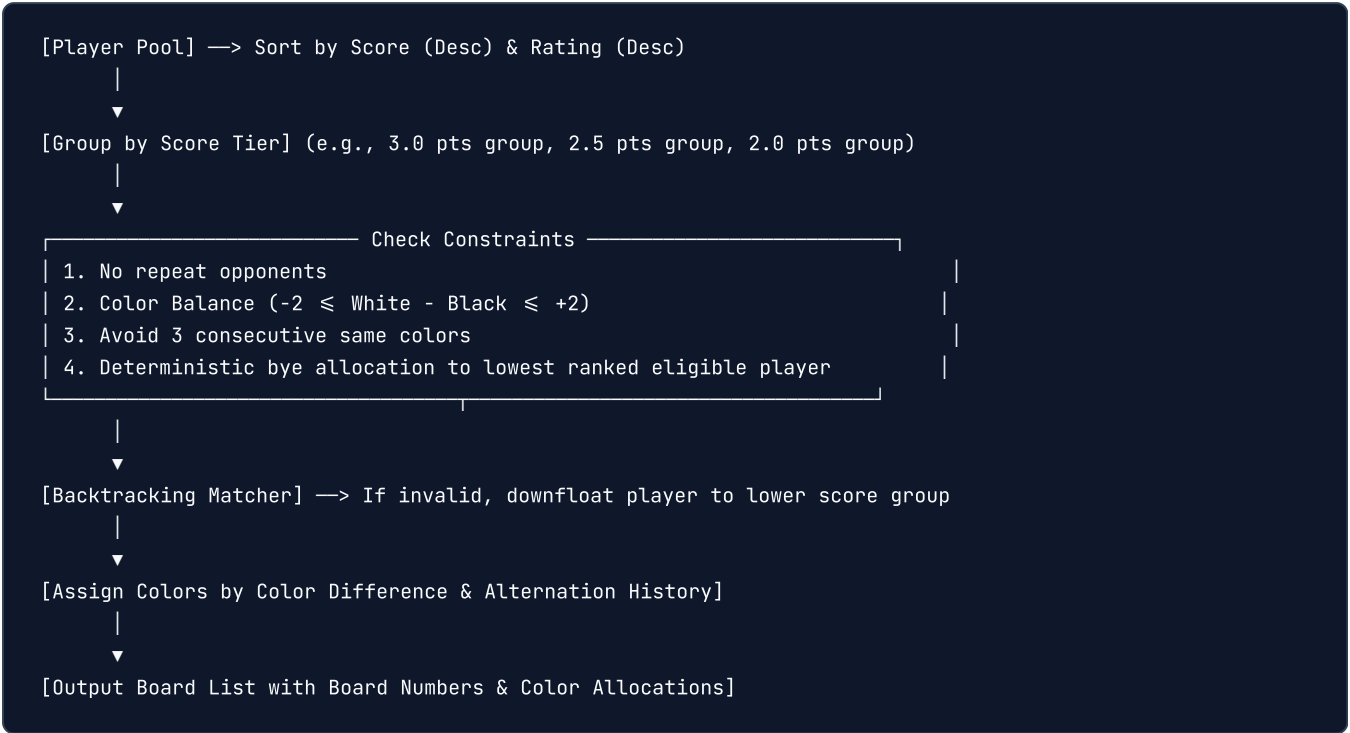
Direct elimination single-elimination bracket for 8, 16, 32, or 64 players. Winners automatically advance to subsequent nodes (Round of 16 → Quarterfinals → Semifinals → Championship Finals).

D. Simplified Arena

Continuous fast-pairing queue logic designed for rapid/blitz time controls. As players finish games, they re-enter the pairing pool and are paired dynamically with the closest available opponent by score.

6. SWISS PAIRING ALGORITHM ARCHITECTURE

The Swiss Pairing Engine is decoupled as a dedicated backend service module (`swissPairing.js`) to ensure mathematical correctness, deterministic behavior, and testability without UI thread blockage:



7. ROUND RESULTS & STANDINGS SYSTEM

7.1 Valid Match Results

- **1 - 0**: White Win (White +1.0, Black 0.0)
- **0 - 1**: Black Win (Black +1.0, White 0.0)
- **½ - ½**: Draw (White +0.5, Black +0.5)
- **1 - 0 (BYE) / ½ - ½ (BYE)**: Unpaired Full-Point / Half-Point Bye (+1.0 / +0.5 score, no rating delta)
- **1 - 0 (Forfeit) / 0 - 1 (Forfeit)**: Opponent No-Show (Counts for score & tiebreaks, excluded from FIDE rating delta)

7.2 Tiebreak Calculation Rules

When scores are tied, rankings are determined by the following hierarchy of tiebreak systems:

TIEBREAK SYSTEM	DESCRIPTION / CALCULATION METHOD
1. Direct Encounter	Head-to-head result between the tied players (applicable if all tied players played each other).
2. Buchholz Cut-1	Sum of opponents' final scores, excluding the single lowest-scoring opponent.
3. Buchholz System	Total sum of scores of all opponents faced throughout the tournament.
4. Sonneborn-Berger	Sum of scores of defeated opponents plus 50% of the scores of drawn opponents.
5. Performance Rating	Calculated rating strength based on opponent average rating and total score percentage.

8. USER INTERFACE LAYOUT SPECIFICATION

The interface uses a clean, focused SaaS layout with high tabular data density, interactive modals, and immediate actionable control items:

♟️ CHESS TOURNAMENT MANAGER [Active: FRCRCE Championship 2026]

Dashboard | Players | Pairings | Leaderboard | Rounds | Settings

[Round: 4 / 7] [Active Players: 48] [Active Games: 24]

CURRENT ROUND PAIRINGS (ROUND 4) [Generate Next Round]

Board	White Player (Rating)	Score	Black Player (Rating)	Result
1	Carlsen, M. (2837)	3.0-3.0	Nakamura, H. (2816)	[1 - 0]
2	Caruana, F. (2779)	2.5-2.5	Gujrathi, V. (2720)	[½ - ½]
3	Erigaisi, A. (2795)	2.5-2.5	Gukesh, D. (2785)	[0 - 1]

TOP STANDINGS: 1. Carlsen, M (4.0 pts) | 2. Gukesh, D (3.5) | 3. Caruana (3.0)

9. FINAL PODIUM DISPLAY SPECIFICATION

Upon round limit completion, the dashboard unlocks the final tournament results and celebratory podium summary:

2

2ND PLACE

H. Nakamura

6.0 Pts

Buchholz: 29.5

1

CHAMPION

M. Carlsen

7.0 Pts

Perf Rating: 2945

3

3RD PLACE

F. Caruana

5.5 Pts

Buchholz: 28.0

RANK	PLAYER NAME	FEDERATION	INITIAL RATING	TOTAL POINTS	PERFORMANCE RATING
1	Magnus Carlsen	NOR	2837	7.0	2945
2	Hikaru Nakamura	USA	2816	6.0	2850
3	Fabiano Caruana	USA	2779	5.5	2810
4	Vidit Gujrathi	IND	2720	5.0	2740

10. SYSTEM ARCHITECTURE & RELATIONAL DATA MODEL

The backend is structured around a relational schema in PostgreSQL managed via Prisma ORM:

```
// PostgreSQL Prisma Entity Schema Overview
Tournament —< TournamentPlayer >— Player
      |
      |—< Round —< Match —< PlayerRoundStats

[Tournament]      : id, name, location, date, type, number_of_rounds, status
[Player]          : id, name, fide_id, rating, age, federation, club, title
[TournamentPlayer]: tournament_id, player_id, starting_rank, score, wins, draws, losses, perf_rating
[Round]           : id, tournament_id, round_number, status (PENDING, ACTIVE, COMPLETED)
[Match]           : id, round_id, board_number, white_player_id, black_player_id, result, is_bye
```

11. REST API SPECIFICATION

HTTP VERB	ENDPOINT PATH	DESCRIPTION / ACTION
POST	/api/tournaments	Create new tournament instance
POST	/api/tournaments/:id/players	Add player to tournament roster
POST	/api/tournaments/:id/rounds	Initialize next tournament round
POST	/api/rounds/:id/pairings	Trigger Swiss / Round Robin pairing engine
PUT	/api/matches/:id/result	Update match result & refresh standings
GET	/api/tournaments/:id/standings	Fetch real-time leaderboards & tiebreaks
GET	/api/tournaments/:id/results	Fetch final tournament results & podium

12. EXECUTION ROADMAP & DEFINITION OF DONE

Phase 1: Core Foundation (MVP)

- Tournament CRUD & Player Registration
- Player roster table with bulk import & editing
- Basic UI Navigation & Manual Pairings
- PostgreSQL Database Schema & Prisma ORM setup

Phase 2: Automated Pairing Engines

- Swiss Pairing Engine with color balancing & backtracking
- Round Robin Berger rotation algorithm
- Tiebreak engines (Buchholz, Sonneborn-Berger, Direct Encounter)
- Live Leaderboard & Standings API

Definition of Done: An organizer can add 50 players, run a 7-round Swiss tournament, enter match results, and generate accurate final standings with tiebreaks without a single manual calculation.